VIDEO LINK

- https://youtu.be/EwC8OzBd4HY

ENHANCEMENT PROPOSAL FOR GEMINI CLI

GROUP #8 – THE TEAMM8TES

TEAM MEMBERS

- **Group Leader:**
  - Christian Fiorino | Proposed Enhancement Description, Potential Risks, Lessons Learnt, Prompt Engineer, AI Report
- **Presenters:**
  - Ananya Kollipara | Testing Plans
  - Lillie Amos | Non-Functional Requirement Effects, Introduction
- **Group Members:**
  - Arlen Smith | Enhancement Implementation Alternative #2, SAAM Analysis, Use Case #2
  - Maia Turner | Impacted Subsystems, High & Low Level Enhancement Effects
  - Vivian Webster | Enhancement Implementation Alternative #1, SAAM Analysis, Use Case #1, Conclusion

AGENDA

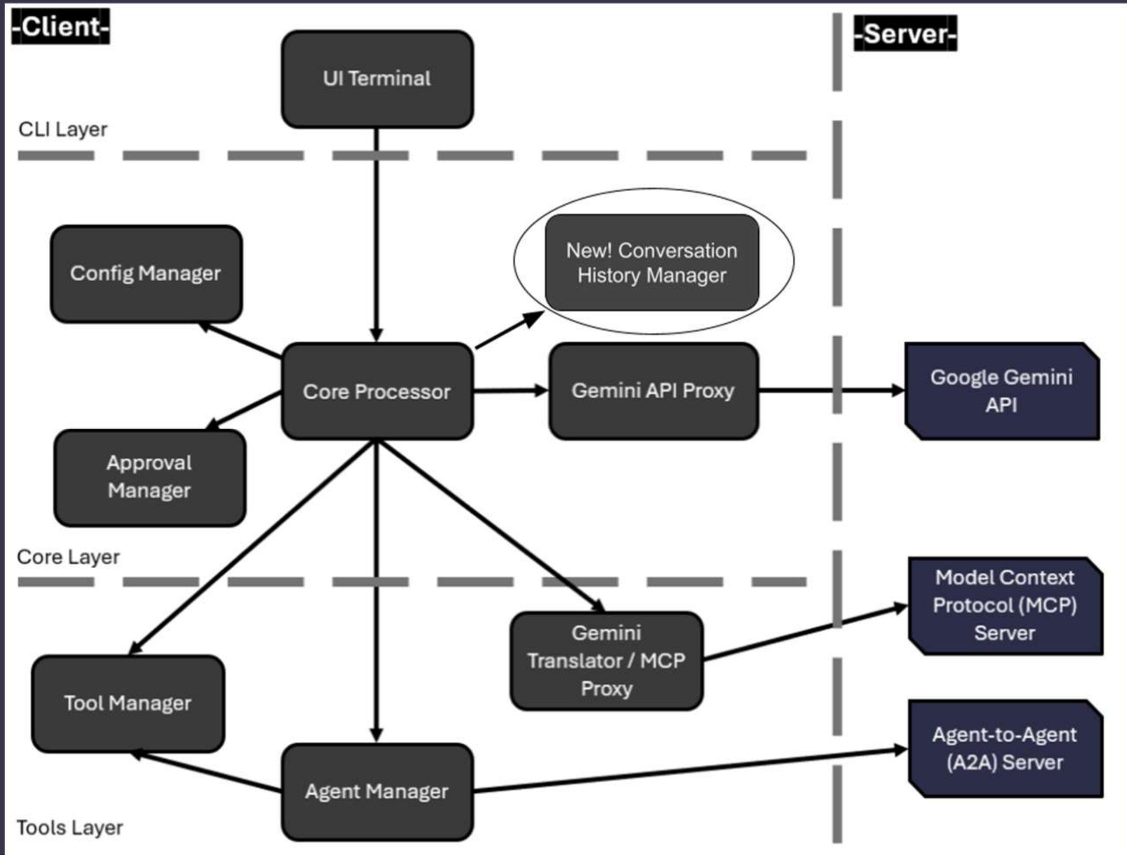
1. Proposed Enhancement
2. Enhancement Implementation Approach #1
3. Enhancement Implementation Approach #2
4. SAAM Analysis
5. Impacted Subsystems
6. Potential Risks & Limitations of Enhancement
7. Testing Plans
8. Use Cases
9. Lessons & Limitations
10. AI Report
11. Conclusion

PROPOSED ENHANCEMENT: CONVERSATION VERSIONING

- Motivated by the lack of a core user experience feature.
- Allows the user to re-prompt any preexisting conversation using any of the available Gemini models.
- Creates a copy of the conversation up until a specified message within it and deletes the rest of the conversation. Allowing users to continue a new conversation from there.
- The user can compare between the original and re-prompted responses.
- Allows users to experiment with the LLM and resolve hallucinations in a streamlined manner.

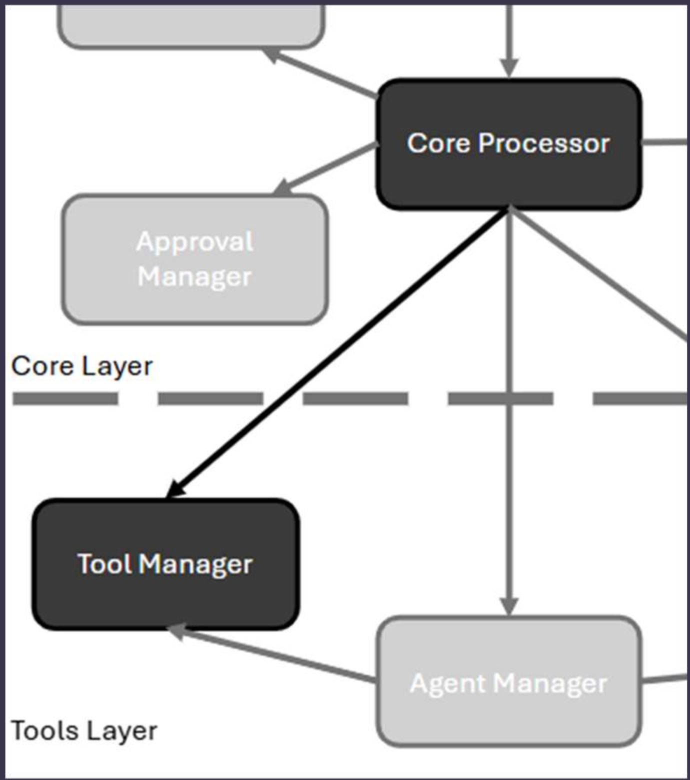
IMPLEMENTATION APPROACH #1

- Introduce a new component that holds prompt history in a tree-like structure
- Adds each prompt + response as a child node to the previous step in the conversation
- User can step back to a previous prompt + response in the conversation and begin a new branch of conversation from there
- New History component overwrites what's in the current conversation context with the information from desired conversation branch



IMPLEMENTATION APPROACH #2

- Introduce a new Git repository managed within the Core Processor component
- Repository would hold conversation history and be updated through git add and git commit commands after every user prompt
- To re-prompt, new tool called the RepromptTool would be defined in the Tool Manager.
- If used, the tool would connect to the Git repository and find a previous user prompt through a git log command.
- Once found, conversation history would be cloned and then the repository would be restored to the point of the original prompt.



SAAM ANALYSIS: END-USERS

Usability:
1, Comes down to how the conversation tree is displayed

2, May be difficult to draw out a specific conversation branch, plus display

Performance:
1, May need a limit on space usage eventually. Very fast relative to other system times. Efficient

2, Space complexity worse since it must store commit details as well. Time complexity roughly the same as 1

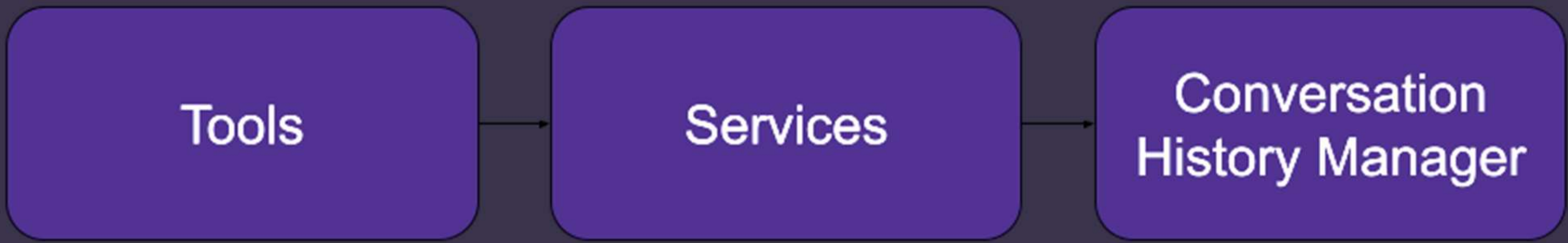
SAAM ANALYSIS: DEVELOPERS

Maintainability:

1, The data structure is incredibly simple, display is all there is to worry about

2, Would rely on a third-party service, and would include unneeded features that could make maintenance less straightforward

IMPACTED SUBSYSTEMS



POTENTIAL RISKS & LIMITATIONS

Performance

- Over time, the number of copied files could reduce memory storage and thus performance.

Maintainability

- If the preexisting session management structure is updated, our feature may become incompatible with the new structure.

Security

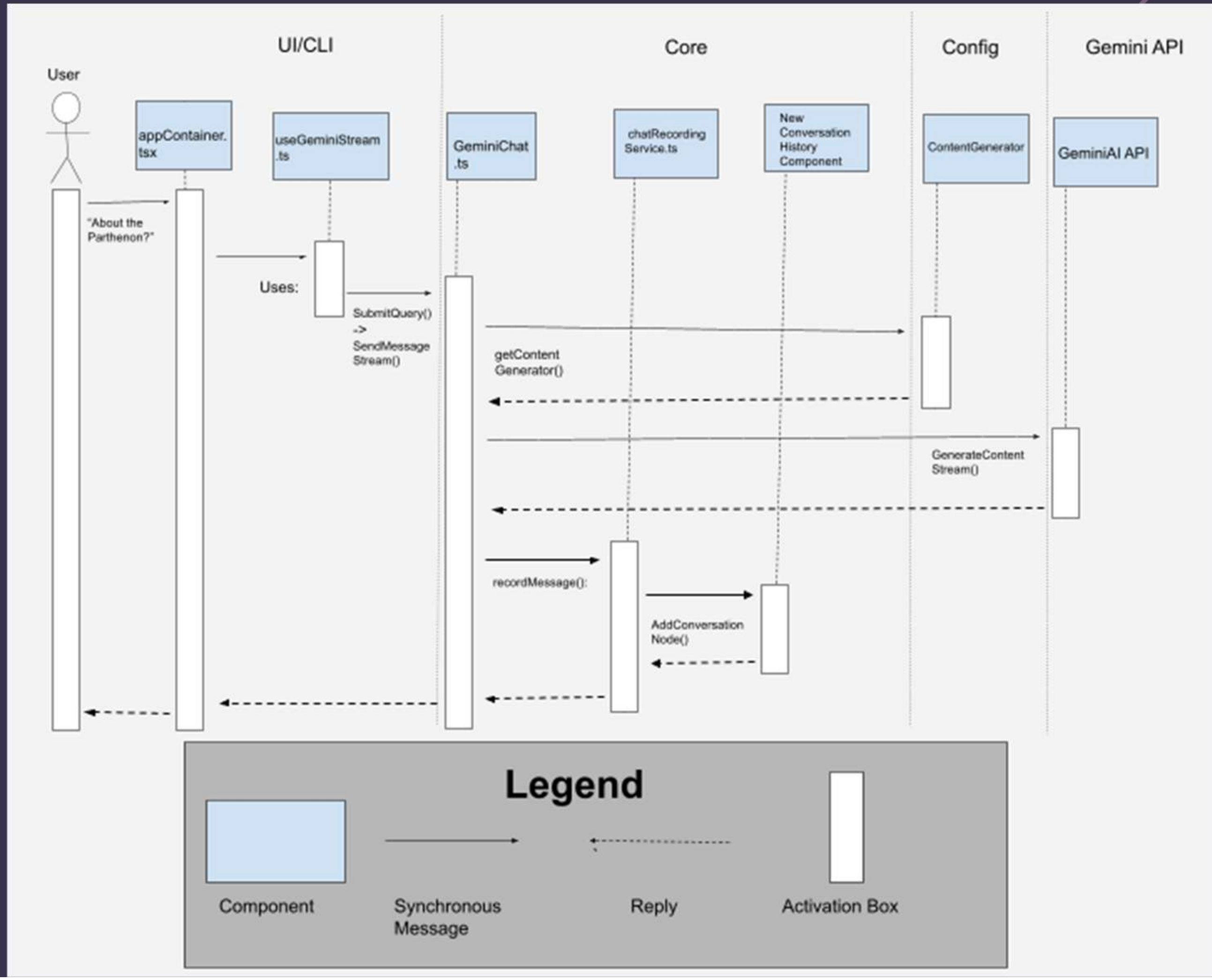
- Lack of encryption on copied conversation files that may contain sensitive information.

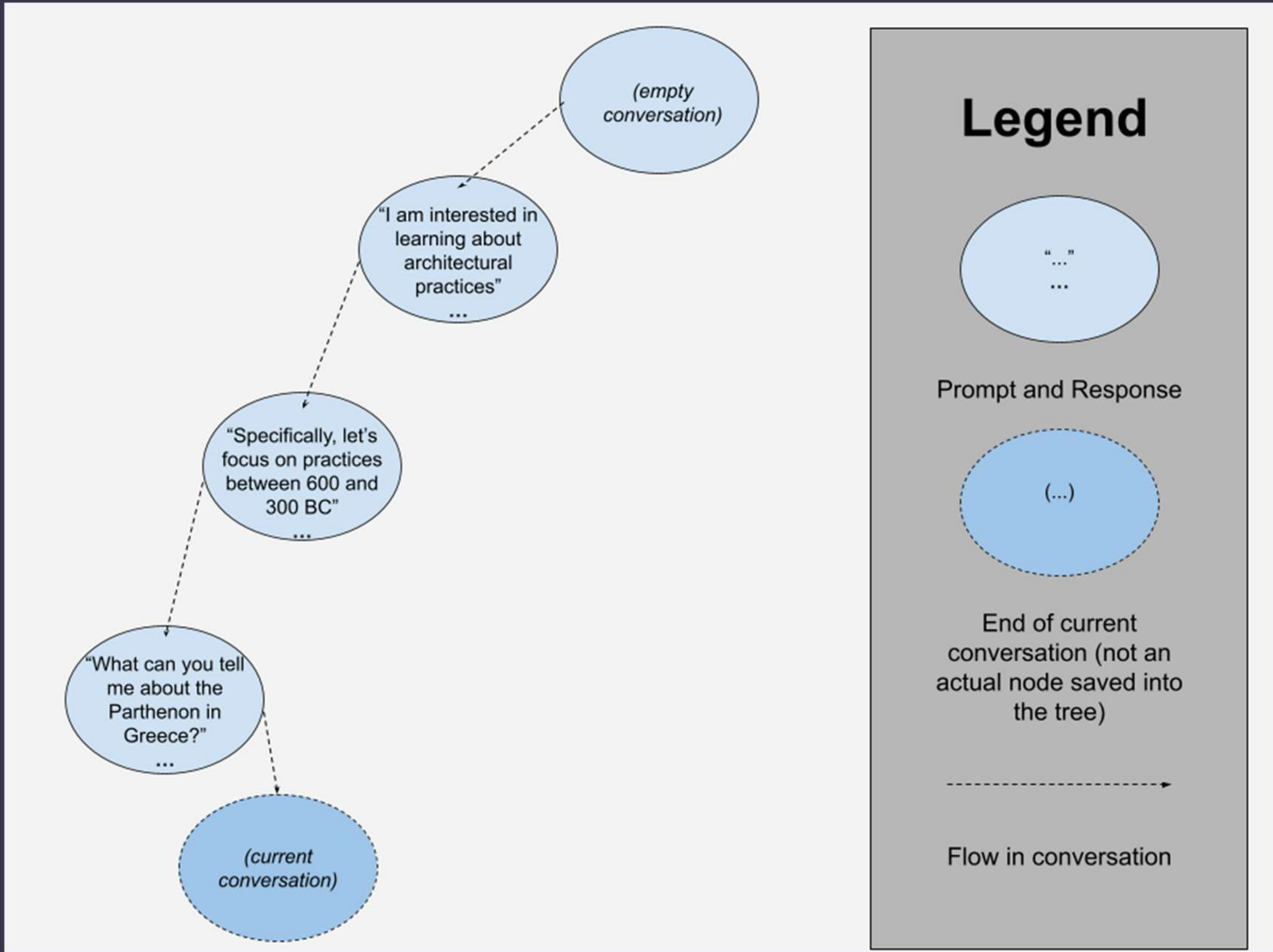
TESTING PLANS

- Unit Testing: Verifies core component functionality of functions, methods & classes
- End-to-end testing: tests prompt flow, encoding and routing between components
- Integration testing: Simulates workflow of mocked endpoints in prompt & response node creation, set up, and placement within the tree
- Regression testing: Verifies no regressions from previous CLI versions, and checks if it produces similar node & tree outputs
- Cross-platform testing: Verifies parsing, encoding and file handling across various OS

USE CASE 1:

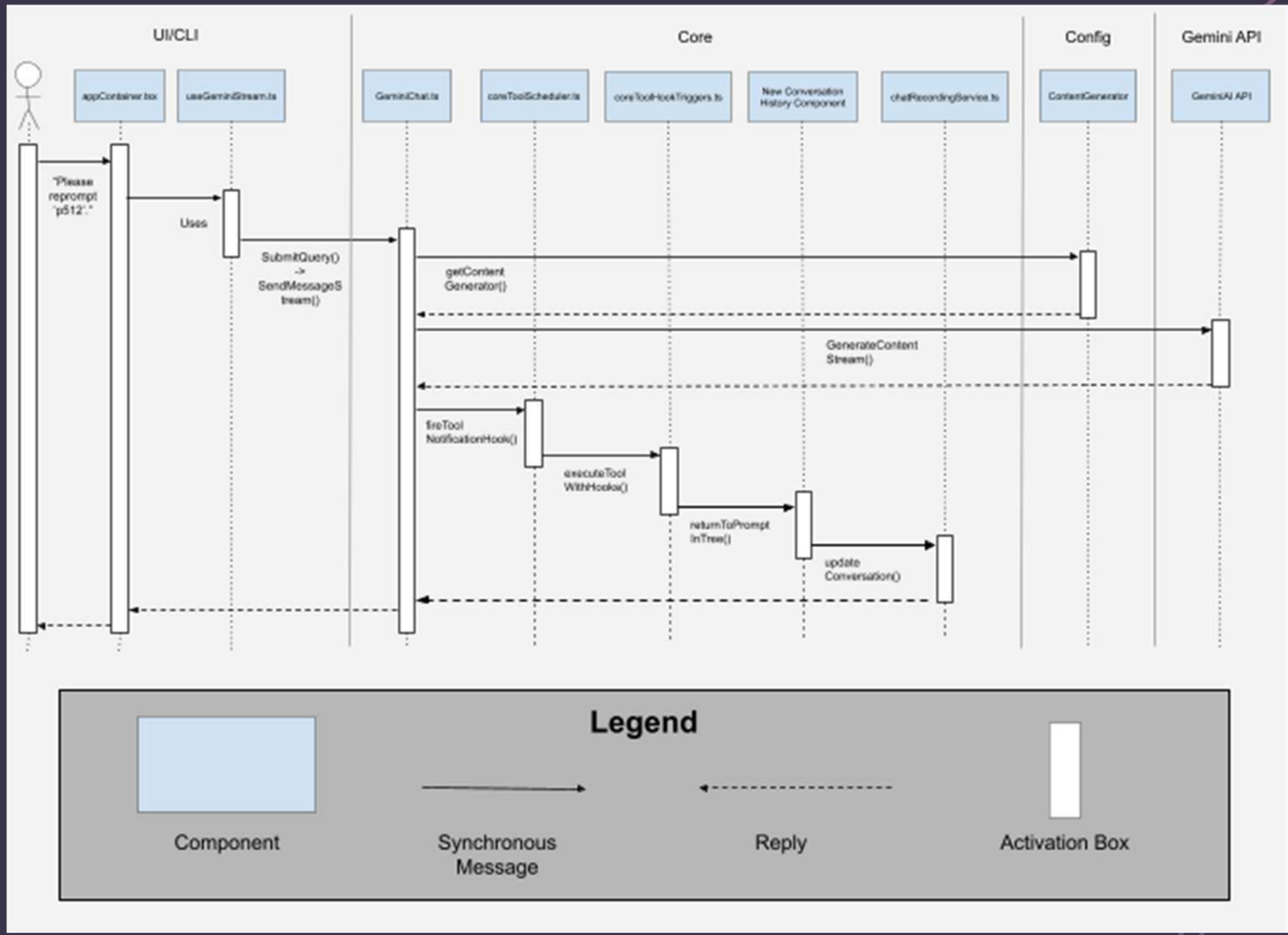
- User prompts Gemini for a response in context of previous conversation
- Response is received
- Prompt + Response is passed through the conversation context component to our new history component
- A new branch is added to the tree off the branch of the previous prompt

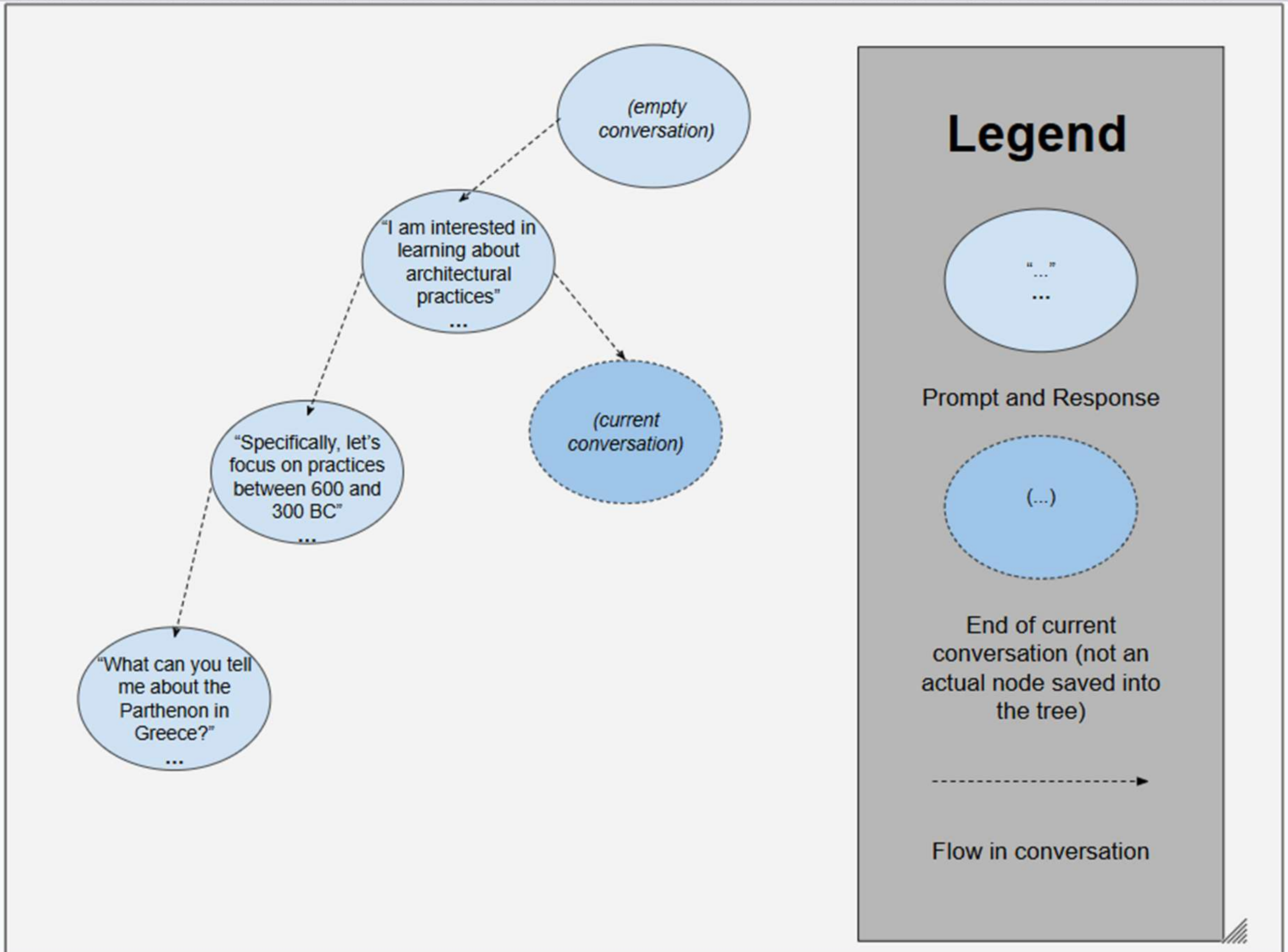



(empty conversation)
"I am interested in learning about architectural practices"
...
"Specifically, let's focus on practices between 600 and 300 BC"
...
"What can you tell me about the Parthenon in Greece?"
...
(current conversation)
Legend
"..."
...
Prompt and Response
(...)
End of current conversation (not an actual node saved into the tree)
Flow in conversation

USE CASE 2:

- User prompts Gemini to re-prompt a previous prompt.
- Response is received
- A tool is used return to the previous prompt saved within the tree structure
- A new branch is added to the tree off the branch of the previous prompt
- The current conversation is updated




(empty conversation)
"I am interested in learning about architectural practices"
...
(current conversation)
"Specifically, let's focus on practices between 600 and 300 BC"
...
"What can you tell me about the Parthenon in Greece?"
...
Legend
"..."
...
Prompt and Response
(...)
End of current conversation (not an actual node saved into the tree)
Flow in conversation

LESSONS & LIMITATIONS

- **Importance of NFRs** – We learned that NFRs are more than just a way to test a system. They act as a vision for a system, ensuring that it has a purpose and achieves some desired goal.
- **Even Work Distributions** – During the project, we realized that we had unevenly split our group's workload. We've now learned the importance of splitting work evenly to create fairer work environments and ensure everyone knows the same knowledge for the assignment.
- There were no limited factors in the assignment.

AI REPORT

- Utilized Gemini v3 Thinking as our virtual member.
- LLM was assigned the tasks of generating potential enhancement ideas and evaluating our report based on the rubric.
- Prompting strategy involved establishing a persona, providing context such as constraints on what ideas the LLM could generate, and cross-referencing the LLM’s response with the official documentation to ensure accuracy and idea uniqueness.
- Despite sufficient idea generation, the LLM did not provide a large impact on group dynamics and deliverables, its contribution rating being (5%).
- Going forward in our endeavors, we will try to be more open-minded and willing to use LLMs despite their inherent risks.

CONCLUSION

- Good Usability, minimal downsides
- Implementation 1 beats out 2 due to reduced bloat
- Good for people using Gemini CLI as a chatbot, helpful as well for those who want a terminal agent
- Should pay mind to security